Projet SDP - Rapport du jalon 1

Apprentissage du modèle MR-Sort par programmation linéaire mixte Modèle retenu et testé - 6 octobre 2026 Rendu attendu sur Edunao : mardi 6 octobre 2026, 23 h 59 (heure de Paris).

Groupe : Mohamed SALH, Mohamed Chebli, Quentin Robidou

1. Objectif et règle de décision

Nous cherchons à apprendre les paramètres d'un modèle MR-Sort à partir d'objets dont les performances et les classes sont connues. Le modèle doit restituer au mieux ces classes. Nous retenons un seul programme : celui qui apprend simultanément les frontières, les poids et le seuil de majorité, en minimisant le nombre d'objets dont les contraintes de classe sont relâchées.

Tous les critères sont à maximiser : une performance plus élevée est préférable. Dans le cas de deux catégories, un objet est accepté si la somme des poids des critères sur lesquels il atteint la frontière est suffisante :

$$s(x) = \sum_{i\,:\,x_i \geq b_i} w_i, \qquad x \in A \Leftrightarrow s(x) \geq \lambda.$$

Sinon, il est refusé. L'égalité à la frontière compte comme un critère satisfait ; l'égalité au seuil de majorité conduit à l'acceptation. Le score est un poids de coalition : on additionne les poids des critères satisfaits, et non les performances elles-mêmes.

Données et constantes

Symbole	Définition
$N = \{1, \ldots, n\}$	Ensemble des critères.
$L = A^* \cup R^*$	Ensemble d'apprentissage ; classes acceptée et refusée connues.
x_i	Performance connue de l'objet x sur le critère i .
ℓ_i, u_i	Bornes connues des performances et de la frontière du critère i .
$\eta > 0$	Petite marge de performance pour représenter le côté strict $x_i < b_i$.
$\varepsilon > 0$	Petite marge de score pour représenter le refus strict $s(x) < \lambda$.
M_i, B	Constantes de relâchement, choisies à partir des bornes.

Variables à apprendre et variables auxiliaires

Variable	Domaine et rôle
b_i	Continue : frontière à atteindre sur le critère i .
w_i	Continue dans $[0, 1]$: poids du critère i .
λ	Continue dans $[0, 1]$: seuil de majorité.
$\delta_i(x)$	Binaire : indique si l'objet atteint la frontière du critère.
$w_i(x)$	Continue : contribution du critère au score de cet objet.
e_x	Binaire : autorise le relâchement des contraintes de classe de l'objet.

Les performances, les classes, les bornes et les marges sont fixées avant la résolution. Seuls b , w et λ constituent les paramètres du classificateur final ; les autres variables servent à formuler l'apprentissage.

2. Programme complet retenu

Le programme suivant est celui retenu pour deux catégories. Il correspond à la formulation fournie, avec les contributions $w_i(x)$ de la question vi.

$$\min_{b,\, w,\, \lambda,\, \delta,\, w(\cdot),\, e} \sum_{x \in L} e_x$$

sous contraintes

$$\sum_{i=1}^{n} w_i = 1$$

$$0 \le w_i \le 1 \qquad \forall i \in N$$

$$0 \le \lambda \le 1$$

$$\ell_i \le b_i \le u_i \qquad \forall i \in N$$

$$b_i - x_i \le M_i(1 - \delta_i(x)) \qquad \forall i \in N,\ x \in L$$

$$b_i - x_i \ge \eta - M_i \delta_i(x) \qquad \forall i \in N,\ x \in L$$

$$0 \le w_i(x) \le w_i \qquad \forall i \in N,\ x \in L$$

$$\delta_i(x) + w_i - 1 \le w_i(x) \le \delta_i(x) \qquad \forall i \in N,\ x \in L$$

$$\sum_{i=1}^{n} w_i(x) \ge \lambda - B e_x \qquad \forall x \in A^*$$

$$\sum_{i=1}^{n} w_i(x) \le \lambda - \varepsilon + B e_x \qquad \forall x \in R^*$$

$$\delta_i(x) \in \{0, 1\} \qquad \forall i \in N,\ x \in L$$

$$e_x \in \{0, 1\} \qquad \forall x \in L$$

Nature du programme et objectif

Toutes les expressions sont linéaires : x_i , M_i , B , η et ε sont des constantes. En particulier, le produit $w_i\delta_i(x)$ est remplacé par les contraintes sur $w_i(x)$. Il s'agit d'un programme linéaire en nombres entiers mixtes (MILP), car des variables continues et binaires coexistent.

L'objectif compte les objets dont on autorise le relâchement. Il pénalise chaque objet de la même façon, quelle que soit l'amplitude de la violation. Si l'objectif vaut zéro, toutes les contraintes de classe, y compris les marges, sont satisfaites. Cette valeur prouve l'optimalité puisque l'objectif ne peut pas être négatif.

Normalisation et bornes

La normalisation des poids fixe leur échelle. Avec des poids positifs ou nuls dont la somme vaut 1, tout score appartient à $[0, 1]$, ce qui explique le domaine du seuil λ . Les bornes $\ell_i \le b_i \le u_i$ maintiennent la frontière dans le domaine des performances et permettent de choisir des constantes M_i valides.

3. Explication des contraintes

3.1. Déterminer si un critère est satisfait

Les deux contraintes sur $b_i - x_i$ imposent le sens de l'indicateur :

$$b_i - x_i \leq M_i(1 - \delta_i(x)), \qquad b_i - x_i \geq \eta - M_i\delta_i(x).$$

Si $\delta_i(x) = 1$, la première devient $b_i - x_i \leq 0$: l'objet atteint bien la frontière, $x_i \geq b_i$. La seconde est inactive si M_i est suffisamment grand.

Si $\delta_i(x) = 0$, la seconde devient $b_i - x_i \geq \eta$: l'objet est en dessous de la frontière, d'au moins η . La première est alors inactive avec un choix valide de M_i .

En particulier, $x_i = b_i$ force $\delta_i(x) = 1$. Une inégalité stricte ne se transmettant pas directement au solveur, $x_i < b_i$ est représenté par $x_i \leq b_i - \eta$. Les positions $0 < b_i - x_i < \eta$ sont donc exclues. Cette marge est une convention du programme testé : pour des performances réelles, elle ne représente pas exactement toutes les comparaisons strictes possibles.

Lorsque x_i et b_i sont dans $[\ell_i, u_i]$, on peut choisir la constante suivante :

$$M_i = u_i - \ell_i + \eta.$$

Le code utilise les mêmes bornes pour tous les critères : $\ell_i = 0$, $u_i = 100$ et $\eta = 10^{-4}$ par défaut ; ainsi $M_i = 100{,}0001$. Même si $e_x = 1$, les contraintes de comparaison restent imposées : seule la classe est relâchée.

3.2. Remplacer le produit par des contraintes linéaires

Le sens recherché est $w_i(x) = w_i\delta_i(x)$. Les contraintes de la question vi le garantissent exactement :

Valeur de l'indicateur	Conséquence des contraintes	Contribution
$\delta_i(x) = 0$	$0 \leq w_i(x) \leq 0$; la borne $w_i - 1 \leq 0$ ne gêne pas.	$w_i(x) = 0$
$\delta_i(x) = 1$	$w_i \leq w_i(x) \leq w_i$.	$w_i(x) = w_i$

Le score $s(x)$ est donc exactement la somme des poids des critères satisfaits. La linéarisation est exacte pour l'indicateur binaire ; la convention de marge concerne les comparaisons, pas ce produit.

3.3. Imposer les classes et permettre les exceptions

Si $e_x = 0$, un accepté vérifie $s(x) \geq \lambda$; un refusé vérifie $s(x) \leq \lambda - \varepsilon$. Si $e_x = 1$, ces contraintes deviennent inactives pour un choix de B suffisant. Comme $s(x), \lambda \in [0, 1]$, le choix du code $B = 1 + \varepsilon$ suffit dans les deux cas. Avec $\varepsilon = 10^{-4}$, on obtient $B = 1{,}0001$.

Un objet relâché peut malgré tout être correctement prédit, par exemple si seul le respect de la marge pose problème. Ainsi, l'objectif compte les relâchements, pas nécessairement les erreurs effectives. Le code enregistre séparément les deux nombres. L'optimalité annoncée porte sur le programme avec les marges fixées.

4. Application à plusieurs catégories et implémentation

Le même modèle pour les catégories ordonnées

Dataset0 contient trois catégories. On applique la même formulation à $p-1$ frontières $b^1, \ldots, b^{p-1}$, qui partagent les mêmes poids et le même seuil. On impose leur ordre sur chaque critère :

$$b_i^h \le b_i^{h+1} \qquad (i \in N,\ 1 \le h \le p-2).$$

Pour une frontière h , les mêmes contraintes de comparaison et de linéarisation définissent $\delta_i^h(x)$ et $w_i^h(x)$. Le score $s_h(x)$ est la somme de ces contributions. Si la classe observée de x est $c(x)$, les contraintes de classe deviennent :

$$s_{c(x)-1}(x) \ge \lambda - Be_x \qquad \text{si } c(x) > 1,$$

$$s_{c(x)}(x) \le \lambda - \varepsilon + Be_x \qquad \text{si } c(x) < p.$$

On conserve une seule variable e_x par objet : une catégorie intermédiaire comporte deux conditions, mais son relâchement ne coûte qu'une unité. Les catégories extrêmes ont une seule condition. L'ordre des frontières implique $s_h(x) \ge s_{h+1}(x)$; les conditions aux autres frontières suivent donc de celles aux frontières adjacentes.

Pour dataset0, b^1 sépare les catégories 1 et 2 ; b^2 sépare les catégories 2 et 3. La catégorie prédite est donnée par :

$$\hat{c}(x) = 1 + \sum_{h=1}^{p-1} \mathbf{1}\{s_h(x) \ge \lambda\}.$$

Il s'agit de l'application du programme retenu à plusieurs frontières, et non d'une autre méthode d'apprentissage.

Organisation du code existant

mrsort.py lit les données avec load_dataset, construit le MILP avec train, puis fournit un modèle MRSortModel pour les scores et les prédictions. Le nombre de critères est déduit de l'en-tête. Le nombre de catégories est déduit de la classe maximale, avec un minimum de deux, ou fixé par l'option --categories.

Le code ne crée les comparaisons qu'aux frontières adjacentes aux classes observées. Il partage aussi les indicateurs et contributions lorsque plusieurs objets ont la même performance pour un critère et une frontière donnés. Ces réductions évitent des variables redondantes sans changer le problème d'apprentissage.

GUROBI résout le MILP ; les variables binaires imposent une recherche entière en complément des relaxations linéaires. Le code laisse le choix des méthodes au solveur. Par défaut : limite de 60 secondes, un thread et graine 0. Les écarts d'optimalité demandés sont nuls ; une limite de temps peut toutefois interrompre la résolution avant la preuve d'optimalité. Le JSON indique explicitement le statut et la présence de cette preuve.

Les tolérances de faisabilité et d'intégralité sont fixées à 10^{-8} . La prédiction utilise une tolérance de 10^{-7} pour les arrondis : $x_i \ge b_i - 10^{-7}$ et $s_h(x) \ge \lambda - 10^{-7}$. Elle reste inférieure aux marges par défaut de 10^{-4} .

5. Comment exécuter le programme

Les commandes suivantes s'exécutent depuis le dossier du projet. Le code et ses dépendances sont fournis dans l'archive ; une licence GUROBI utilisable doit être disponible. La version fixée dans requirements.txt, et vérifiée pour ce rendu, est gurobipy==13.0.3.

Installation et apprentissage sur dataset0

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python mrsort.py dataset0.txt --log \
  --output resultat_dataset0.json \
  --export-lp modele_dataset0.lp
```

La dernière commande affiche le résultat, écrit les paramètres et diagnostics dans le JSON, et exporte le programme linéaire dans le fichier LP. Les valeurs rapportées dans les pages suivantes proviennent des fichiers de résultats inclus dans l'archive.

Format d'un autre jeu de données

```
id   crit1   crit2   categ.
x1   65      40      2
x2   20      15      1
```

Chaque ligne contient un identifiant, les n performances puis la classe entière de 1 à p . Les catégories vont de 1, la plus basse, à p , la plus haute. Les espaces et tabulations sont acceptés. Si une catégorie est absente du jeu d'apprentissage, préciser le nombre total avec --categories.

```
.venv/bin/python mrsort.py mon_jeu.txt --categories 2 \
  --lower-bound 0 --upper-bound 20 \
  --output resultat_mon_jeu.json
```

Tests et protocole d'évaluation

```
.venv/bin/python -m unittest -v test_mrsort
.venv/bin/python benchmark.py --runs 20 \
  --criteria 3 --categories 2 --train-size 30 \
  --test-size 200 --seed 42 --time-limit 10 \
  --output resultat_benchmark.json
```

Les neuf tests existants vérifient notamment la lecture du fichier, les égalités aux seuils, l'ordre des catégories, la restitution de jeux compatibles et le traitement d'objets identiques portant des classes contradictoires. Ils passent lors de la vérification de ce rendu.

Réponses courtes sur l'utilisation

Que renvoie le programme ? Les poids, les frontières, le seuil, les prédictions, la précision d'apprentissage, les objets relâchés ou mal classés, le statut et les diagnostics du solveur.

Comment changer les marges ou le temps disponible ? Utiliser --performance-margin, --score-margin et --time-limit. Les marges valent 10^{-4} par défaut ; leur modification change le problème résolu.

Comment réutiliser les paramètres ? Charger le champ model du JSON avec MRSortModel(**result["model"]), puis appeler predict sur un vecteur de performances dans l'ordre des critères.

6. Résultats obtenus sur dataset0

Dataset0 contient 60 objets, 5 critères et 3 catégories, avec des performances dans $[0, 100]$. Les effectifs sont : 19 objets en catégorie 1, 25 en catégorie 2 et 16 en catégorie 3. La solution enregistrée dans resultat_dataset0.json fournit :

Paramètre	crit1	crit2	crit3	crit4	crit5
Poids w_i	0,0001	0,3333	0,3332	0,0001	0,3333
Frontière b_i^1	33,0001	36	37	35	24,0001
Frontière b_i^2	41	53	57	60,0001	43

Seuil de majorité : $\lambda = 0{,}6667$. Les décimales du tableau sont présentées à quatre chiffres ; le JSON contient les valeurs extraites du solveur avec leur précision numérique.

Indicateur	Résultat enregistré
Statut GUROBI	OPTIMAL
Objectif / borne de l'objectif	0 / 0
Écart d'optimalité	0
Objets relâchés / mal classés	0 / 0
Précision d'apprentissage	60/60, soit 100 %
Taille du MILP construit	742 variables ; 1 756 contraintes
Temps de résolution enregistré	Environ 0,491 seconde

Le temps correspond à la mesure du solveur enregistrée, pas à toute l'exécution du script. Il dépend de la machine. Une nouvelle exécution de contrôle retrouve les mêmes paramètres et les mêmes classes ; seul le temps varie.

Courte analyse

L'objectif nul et la précision de 100 % établissent que le modèle retenu restitue parfaitement ce jeu d'apprentissage. Les frontières sont ordonnées et toutes les marges de classe sont satisfaites. La solution respecte donc le critère d'apprentissage et la validation sur dataset0.

Les critères 2, 3 et 5 portent presque tout le poids, mais les poids de crit1 et crit4 ne doivent pas être supprimés par arrondi : ils peuvent faire atteindre exactement le seuil. Les paramètres appris ne sont pas nécessairement uniques. Enfin, cette restitution parfaite ne mesure pas à elle seule la qualité sur de nouveaux objets ; l'évaluation indépendante ci-après répond à cette question.

7. Évaluation sur vingt jeux aléatoires

Protocole du jalon 1

Le script benchmark.py suit le protocole demandé. Pour chacune des 20 répétitions, il génère un modèle de référence M' , puis 30 objets d'apprentissage classés par ce modèle. Le MILP apprend un modèle M^* . Ensuite, 200 nouveaux objets sont générés, sans être transmis au solveur ; on compare les classes prédites par M' et M^* .

La configuration enregistrée utilise 3 critères et 2 catégories. Les performances sont tirées uniformément dans $[0, 100]$, les frontières dans $[10, 90]$, les poids positifs sont tirés puis normalisés, et le seuil de majorité est tiré dans $[0{,}5, 0{,}85]$. Les graines vont de 42 à 61 ; la limite de résolution est de 10 secondes par apprentissage.

La qualité d'une répétition est la proportion de classes identiques sur le test indépendant :

$$q = \frac{1}{|T|} \sum_{x \in T} \mathbf{1}\{M'(x) = M^*(x)\}.$$

Résultats enregistrés

Mesure	Valeur sur les 20 répétitions
Précision moyenne sur l'apprentissage	100 %
Résolutions optimales / objectif nul	20/20 ; objectif 0 à chaque fois
Précision moyenne sur les objets de test	93,025 %
Écart-type descriptif des précisions	3,576 points de pourcentage
Précisions minimale / maximale	82 % / 96,5 %
Accord cumulé sur les tests	3 721 objets sur 4 000
Temps moyen de résolution enregistré	Environ 0,0285 seconde

Courte analyse des performances

Les 20 apprentissages restituent parfaitement leurs 30 exemples, tandis que l'accord moyen sur des objets indépendants est de 93,025 %. Le modèle apprend donc une règle proche du modèle de référence, mais une restitution parfaite de l'apprentissage ne garantit pas une généralisation parfaite. Avec seulement 30 exemples, plusieurs modèles peuvent expliquer les mêmes classes et différer ailleurs.

Les écarts entre répétitions indiquent une sensibilité au modèle et aux objets tirés. L'écart-type rapporté décrit ces 20 essais ; il ne constitue pas un intervalle de confiance. Ce benchmark porte sur deux catégories : la prise en charge de trois catégories est validée séparément sur dataset0, mais sa précision de généralisation n'est pas mesurée ici. Les poids appris n'ont pas à être identiques à ceux de référence pour obtenir les mêmes classes.